Base de Datos

Universidad Tecnológica del Perú

Inicio

Logro de aprendizaje

Al término de la sesión, el estudiante comprenderá la importancia del manejo de transacciones en SQL Server y aplicará los comandos BEGIN TRANSACTION, COMMIT y ROLLBACK para mantener la integridad y coherencia de los datos.

Universidad Tecnológica del Perú

Alguna duda de la clase anterior

¿Recordemos lo desarrollado en la clase anterior?

Utilidad

La utilidad de una transacción en SQL Server, es garantizar la integridad, consistencia y seguridad de los datos, especialmente cuando se realizan varias operaciones relacionadas entre sí.

UTP Universidad Tecnológica del Perú

Transacciones

Transformación

Transacción es un conjunto de instrucciones SQL que se ejecutan de manera atómica, para garantizar la integridad de los datos en la base de datos.

https://www.youtube.com/watch?v=HsAsyo6Lxx8

Transacciones

Transformación

Principios ACID

Las transacciones siguen los principios ACID, que garantizan su fiabilidad:

Principio	Significado	Qué asegura
A – Atomicidad	Todo o nada	Si falla una parte, se deshace todo.
C – Consistencia	Estado válido a válido	Los datos siempre quedan coherentes.
I – Aislamiento	Independencia	Una transacción no afecta a otra mientras se ejecutan.
D – Durabilidad	Permanencia	Una vez confirmada (COMMIT), los cambios no se pierden.

Transacciones

Comandos principales para manejar transacciones

- **BEGIN TRANSACTION**: Inicia una nueva transacción.
- **COMMIT**: Confirma la transacción, haciendo permanentes todos los cambios.
- **ROLLBACK**: Deshace todas las operaciones de la transacción, volviendo a su estado original.

Comando	Significado	Función
BEGIN TRANSACTION	Inicia la transacción	Marca el punto de inicio.
COMMIT TRANSACTION	Confirma los cambios	Guarda los datos definitivamente.
ROLLBACK TRANSACTION	Cancela la transacción	Revierte los cambios realizados.

Sintaxis:

```
CREATE PROCEDURE proc_nom (/*parám */
) AS
BEGIN
        BEGIN TRANSACTION
        SAVE TRANSACTION /* punto salvado */
        /* declaración y asignación de variables*/
        BEGIN TRY
                /* Código del procedimiento */
                COMMIT TRANSACTION
        END TRY
        BEGIN CATCH
                IF @@TRANCOUNT > 0
                        BEGIN
                                ROLLBACK TRANSACTION /* punto salvado */
                        END
        END CATCH
END;
```

- ✓ **BEGIN TRANSACTION;**: Comienza la transacción.
- ✓ **BEGIN TRY … END TRY**: Bloque para ejecutar las operaciones dentro de la transacción.
- ✓ **COMMIT**: Si no hay errores, confirma la transacción y hace permanentes los cambios en la base de datos.
- ✓ **BEGIN CATCH … END CATCH**: Bloque para manejar los errores. Si ocurre un error dentro de la transacción, se ejecuta el código en el bloque CATCH.
- ✓ **ROLLBACK**: Si hay un error, deshace todos los cambios realizados hasta el momento dentro de la transacción.

Ejemplo de utilidad práctica

Supongamos que una tienda registra una venta:

1. Resta el stock del producto.
2. Inserta la venta en la tabla Orders.
3. Inserta los detalles en OrderDetails.

Si alguna de las tres operaciones falla, se revierte todo con ROLLBACK, garantizando que la base de datos permanezca coherente.

Utilidad	Descripción
Integridad	Asegura que los datos no queden incompletos
Consistencia	Mantiene la coherencia de la información.
Recuperación	Permite deshacer cambios ante errores.
Seguridad	Protege los datos durante operaciones críticas.
Concurrencia	Evita conflictos cuando varios usuarios trabajan al mismo tiempo.

```sql
BEGIN TRANSACTION;  -- Inicia la transacción

BEGIN TRY
    -- Primer comando de la transacción: Insertar un nuevo registro en la tabla de empleados
    INSERT INTO Empleados (Nombre, Apellido, Departamento)
    VALUES ('Juan', 'Pérez', 'Ventas');

    -- Segundo comando de la transacción: Actualizar un registro en otra tabla
    UPDATE Departamentos
    SET NumeroEmpleados = NumeroEmpleados + 1
    WHERE Nombre = 'Ventas';

    -- Si ambas operaciones son exitosas, hacemos un commit para hacer permanentes los cambios
    COMMIT;
END TRY
BEGIN CATCH
    -- Si ocurre un error, hacemos un rollback para deshacer todos los cambios realizados en la transacción
    ROLLBACK;

    -- Opcionalmente, puedes manejar el error (por ejemplo, mostrar el mensaje del error)
    PRINT 'Ocurrió un error y se ha deshecho la transacción: ' + ERROR_MESSAGE();
END CATCH;
```

```sql
BEGIN TRANSACTION; -- Iniciar la transacción

-- Insertar un nuevo pedido en la tabla Orders
INSERT INTO Orders (CustomerID, EmployeeID, OrderDate, ShipName,
ShipAddress, ShipCity, ShipRegion, ShipPostalCode, ShipCountry)
VALUES ('ALFKI', 1, GETDATE(), 'Empresa de Envío', '123 Calle
Principal', 'Ciudad Envío', 'Región Envío', '12345', 'País
Envío');

-- Obtener el ID del último pedido insertado
DECLARE @OrderID INT;
SET @OrderID = SCOPE_IDENTITY();

-- Insertar detalles del pedido en la tabla Order Details
INSERT INTO [Order Details] (OrderID, ProductID, UnitPrice,
Quantity, Discount)
VALUES (@OrderID, 1, 18.00, 10, 0.05); -- Supongamos que
ProductID 1 representa el producto en el pedido

-- Confirmar o deshacer la transacción
IF @@ERROR = 0
BEGIN
    COMMIT TRANSACTION; -- Confirmar la transacción si no hay
errores
    PRINT 'La transacción se completó con éxito. Se insertó un
nuevo pedido y sus detalles.';
END
ELSE
BEGIN
    ROLLBACK TRANSACTION; -- Deshacer la transacción en caso de
error
    PRINT 'Error durante la transacción. Se deshicieron todas las
inserciones.';
END
```

Práctica

Conclusiones

- Los procedimientos almacenados ejecutan una serie de sentencias, pero no devuelven ningún valor. Sin embargo pueden generar una consulta.
- Las funciones pueden devolver valores o tablas que se pueden usar en consultas mediante el SELECT.
- Las transacciones contienen internamente COMMIT y ROLLBACK permiten confirmar o cancelar una transacción respectivamente.

Universidad
Tecnológica
del Perú